Sensor Trust for Lunar South Pole Landing

Project description - LANTERN simulation testbed

A lunar lander's navigation filter should trust each sensor according to the terrain and lighting ahead, so that a real cliff is not mistaken for a broken sensor and a blinded camera is not believed. The project tests this by landing a simulated lander on real NASA South Pole terrain.

1. The problem

During powered descent a lander fuses a laser altimeter, an optical camera and an IMU. With a fixed outlier gate and fixed sensor weights the filter cannot tell a real terrain discontinuity, or a shadowed featureless scene, from a sensor fault. It then either rejects valid altimeter data at crater rims (the HAKUTO-R M1 failure, April 2023) or keeps trusting a camera that cannot see in shadow (a contributor to IM-2, March 2025). The question is how to adjust, ahead of time and along the planned trajectory, how much the filter trusts each sensor, using known terrain and illumination.

2. Proposed solution: context-aware sensor trust layer

For every altimeter reading the trust layer asks four questions in order:

- â€¢ **Is it physically possible?** Find the lowest and highest ground on the map within the area where the lander might be. A reading that fits nowhere on the map is a fault and is rejected.
- â€¢ **Probation.** A surprising but possible reading is neither believed nor rejected at once. For 2-3 s the lander watches the next readings; if they agree with each other and follow the map's shape they are accepted and the height is reset, otherwise they are rejected.
- â€¢ **Camera and shadow.** The map and Sun position say whether the ground is lit. In darkness the camera is not used; in partial light it is used with less weight.
- â€¢ **No map coverage.** Outside the map the standard fixed rule is used with probation, in a more cautious mode.

The trust layer adds map-driven terms to the standard chi-square gate: the variance of DEM heights inside the position-uncertainty ellipse, and the LOLA height error from the PGDA error map. Seven ways of judging a surprising reading are compared on identical descents: fixed rule (baseline), PITFAL-style, map tolerance, wait-and-see, second opinion, ML classifier and the full four-step rule.

3. Data

NASA PGDA Product 78 (Barker et al.), Site04 Shackleton rim: a 16 x 16 km tile at 5 m/pixel (3200 x 3200), spanning -2848 m to +1805 m (4.65 km of relief), slopes up to about 70 degrees. Along one track across the rim the ground falls 4.5 km over about 9 km. With the Sun 2 degrees above the south-east horizon at least 51% of the tile is in cast shadow, and another 46% cannot be classified from this tile alone because terrain up to about 130 km away can cast shadows. The slope GeoTIFF supplied in *data/* is truncated, so slope is derived from the elevation file. Site01 is not present in this repository.

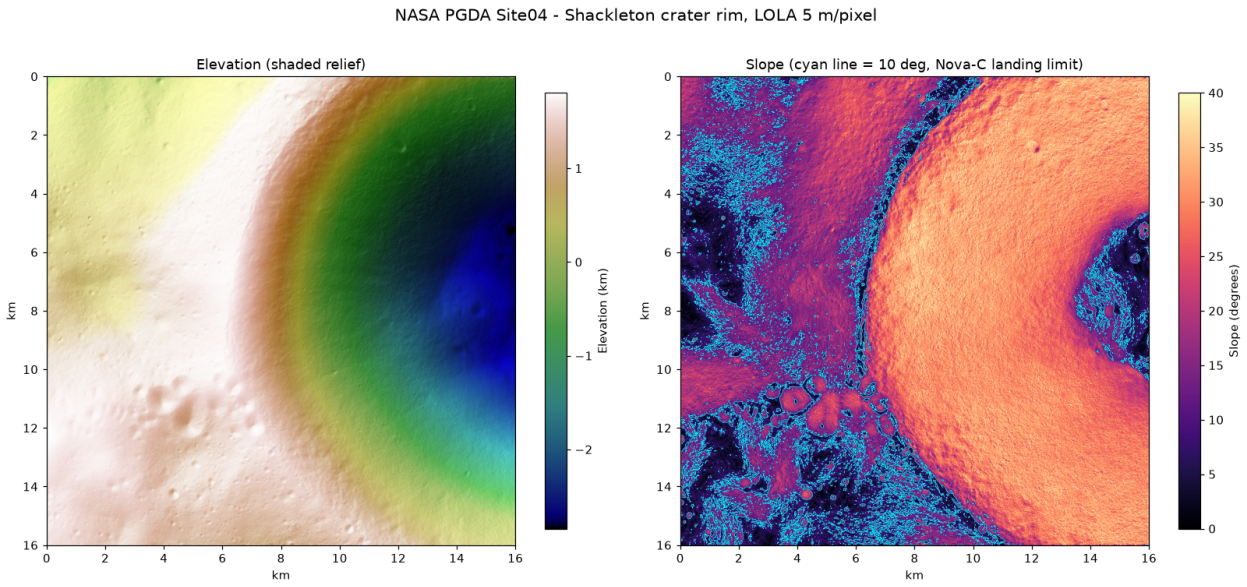


Figure 1. Site04 elevation (left) and slope (right).

4. System architecture

Layer	What it does	Where
Terrain	PGDA DEM converted to a Unity heightmap (2049 px, 16 km); procedural crater worlds for tuning and training	LANTERN/Assets/Terrain, tools/make_heightmap.py, sim/world.py
Lighting	Directional Sun at 1.5-3 degrees with cast shadows; shadow rasters from the DEM	Site04SceneBuilder.cs, tools/render_site.py
Physics (Unity)	Rigid-body lander, lunar gravity 1.62 m/s2, throttleable engine, fuel with Isp 300 s, legs	LanderController.cs
Guidance (Unity)	Descent autopilot: free-fall coast, speed-profile braking, upright hold; optional time warp	DescentAutopilot.cs
Live scan (LUMEN)	64x64 lidar raycasts, height patch, match to stored map, position correction with confidence; patch cell grows with altitude	Assets/Scripts/Lumen/
Data logging	One CSV per run plus a one-line summary per run	TelemetryLogger.cs -> out/runs/
Fast test bench (Python)	Batched simulator for hundreds of seeded descents: sensors, faults, EKF, judges, scenarios	sim/engine.py, sim/world.py, sim/pipeline.py
Replay (Unity)	Side-by-side replay of recorded landings with a ghost at each filter's estimate	ReplayPlayer.cs, ReplaySceneBuilder.cs

5. Objectives and experiments

#	Objective (from the project plan)	Status
O1	Hazard (cliff) and blind-spot (shadow) maps for Site04 and Site01	Done inside the test bench: cliff (terrain-height spread) and shadow (lit-fraction) statistics from the DEM and Sun position
O2	Trust layer producing gate width and measurement noise, with fault-vs-terrain check	Done and evaluated: sim/engine.py (500 runs per scenario)
O3	Reproduce baseline failures (cliff, shadow, delay, dropout)	Done: cliff failure reproduced (fixed rule fails 99.6% of runs); shadow, delay and dropout scenarios run
O4	Compare baseline, PITFAL-style and trust layer over ~500 Monte Carlo runs per scenario	Done: 500 runs per cell, 11 judges and ablations, 95% intervals
O5	Release Unity project, code, scenarios and a side-by-side replay	Done: Site04Replay scene plays a recorded descent with all three filters side by side
O6	Site-change robustness (unseen site, divert, wrong or missing map)	Done: unseen site, mid-descent divert, shifted map, missing map

#	Objective (from the project plan)	Status
O7	Live 3D scan extension with three-way check	Done: LUMEN in Unity; scan fix and three-way check in the test bench

Metrics: landing success (upright within speed and tilt limits), altitude error (RMS), false rejection of valid altimeter readings, missed injected faults, camera weight in shadow, fuel left at touchdown, and success on an unseen site compared with the tuned site. Landing success here means: vertical speed at most 2 m/s, horizontal speed at most 1.5 m/s, ground slope at most 10 degrees, fuel remaining.

6. Fair-testing rules (fixed before any experiment)

- â€¢ The onboard map differs from the simulated Moon (unmapped craters and roughness, small registration shifts, height noise).
- â€¢ Count both mistakes: real terrain wrongly rejected and faults wrongly believed.
- â€¢ Include faults exactly at cliffs and faults together with cliffs.
- â€¢ Tune and train on procedural terrain only; test on Site04, never used for tuning.
- â€¢ About 500 runs per scenario with fixed seeds, reporting success with 95% intervals.
- â€¢ Every method sees the same descents, noise and faults (common random numbers).
- â€¢ The hundreds of landings run in the fast Python bench; Unity replays selected landings.

7. Scope and assumptions

In scope: powered descent to touchdown with a laser altimeter, optical camera and IMU, on real DEMs plus procedural terrain. Out of scope: orbit and deorbit, flight hardware, certification, boulder avoidance, engine failures, sensors that never switch on, plume dust (planned later extension). Assumptions: the planned trajectory is known before descent; the DEM's stated height error bounds map error.

8. Known limits stated up front

- â€¢ A fault that is a constant offset lasting longer than the probation window looks like terrain and passes probation.
- â€¢ Delayed altimeter readings carry a steady offset; probation is expected to help only partly.
- â€¢ A dead altimeter (IM-1) is not solved by trust logic.
- â€¢ The simulated camera model and the 'real Moon' roughness are simplified stand-ins.